

Method Validation: HITL Edge Processing

Literature check on the testbed design, with citable sources. Searched August 2026.

Verdict

The approach follows established practice. ns-3 TapBridge is the documented mechanism for attaching physical hardware to an emulated network, and three published systems perform the same ns-3 + Gazebo + ROS co-simulation.

"TapBridge allows a real host to participate in an ns-3 simulation as if it were one of the simulated nodes" — ns-3 Model Library [1]

This is exactly the mechanism by which the Raspberry Pi occupies aircraft node 2 of the scenario.

Prior work worth citing

System	What it does	Relation to this work
CORNET [3]	Connects ns-3 to Gazebo for robot networks	Same pairing as the baseline stack
FlyNetSim [4]	Synchronises ns-3 with ArduPilot	Close to the SITL arrangement used here
FANS [5]	ns-3 + ROS + Gazebo, extended to embedded hardware (Jetson Nano + Pixhawk 4)	Nearest match: also puts a real board in the loop

FANS is the strongest comparison point. It does the same thing — a real embedded board in the loop — but uses an NVIDIA Jetson Nano rather than a Raspberry Pi 4B. That hardware difference is the main methodological distinction to acknowledge.

What is distinctive in this work

No published description was found of **separating the sensor and radio paths** in this context. The co-simulation frameworks above route traffic through the emulated network without distinguishing which links would physically be radios. The VLAN-based split used here is therefore a contribution, and should be stated as such rather than presented as routine setup.

A second justification for the split

The ns-3 documentation records a real limitation of real-time mode:

"ns-3's real-time mode cannot cope with large and complex networks, and its accuracy decreases with higher traffic volumes or particularly bursty traffic" — Performance Evaluation of ns-3 Real-Time Emulation [6]

The camera stream is 110 Mbps at the delivered 5 Hz, and 481 Mbps at the 20 Hz the sensor originally ran at. Either figure is far outside the regime in which ns-3 real-time emulation stays accurate. Had imagery crossed the emulated channel, the channel model itself would have become a source of error. Passing only 118-byte messages through it — a reduction of roughly 23,000x per frame — keeps the testbed inside the accurate operating region.

The two-link split is not only physically faithful — it is required for the emulation to be trustworthy. This is a stronger argument than the physical one alone, and it is citable.

Measured results agree with published work

"the RPI5 failed to satisfy the real-time processing needs in spite of its suitability for low-energy consumption applications" — YOLO on a standalone Raspberry Pi 5 [7]

"On Raspberry Pi 5, CPU-only execution of large models is impractical due to multi-second per-frame latency" — YOLOv8 / RT-DETR energy efficiency on edge devices [8]

The figures measured here sit exactly where that literature predicts: 2,540 ms with PyTorch at 960x544, approximately 1,015 ms in the live pipeline at 640, and approximately 1,000 ms with NCNN at 960x544 — all on a Pi 4B, which is slower than the Pi 5 used in both studies. The results are therefore consistent with independent work rather than anomalous, and the report can say so.

Detection quality is separately validated against a known ground truth of five human models at fixed world coordinates: 1 to 4 detected per frame at confidences of 0.42 to 0.59, with the shortfall attributable to mutual occlusion at approximately one metre spacing rather than to detector deficiency. Because the subject count is known exactly, recall is counted rather than estimated, and any count above five would be a false positive without adjudicating individual boxes.

The same body of work also characterises the offloading side, reporting end-to-end latency rising from 0.123 s to 2.317 s as available bandwidth falls from 1 Mbps to 50 Kbps [9] — the curve the ground arm of this experiment will trace.

Instrumentation and measurement validity

A co-simulation testbed of this kind fails quietly. The measurements below are reported with the checks that establish they mean what they are claimed to mean, because in this system a fault at a low layer does not raise an error at the layer above — it produces plausible and wrong behaviour there.

Check	Why it is necessary	Observed
Measure rate at the source as well as the destination	Distinguishes a slow producer from a lossy transport in one step	Host 5.00 Hz while the edge node saw 0.19 Hz — localised the fault to transport immediately
Verify socket buffers in BOTH directions	A clamped send buffer discards fragments before the NIC; no interface counter records it	wmem_max left at 208 kB silently capped a 16 MB DDS send buffer; tx_dropped and rx_errors were 0 throughout
Confirm the simulation clock is advancing	Lockstep physics means a stalled autopilot freezes time, and every derived rate becomes meaningless while staying plausible	Real-time factor 0.998, so rates are trustworthy
Time inference around the model call alone	Excludes queuing, so a slowdown is attributable to CPU contention rather than to waiting	623 ms standalone against the in-pipeline figure
Confirm detections visually before accepting counts	A detector boxing ground texture satisfies every numerical check	An earlier configuration reported 16 confident detections against 5 subjects, none correct

The send-buffer fault is the clearest instance. Its visible symptoms were that the detector found no humans in 93 of 94 frames and that the patrol mission timed out at every waypoint — both of which invite a detector or flight-control explanation, and neither of which was the cause. The aircraft flew correctly and the detector functioned correctly; the scene was simply being observed through roughly one frame every five seconds.

Optimisation options

Option	Expected effect	Cost
NCNN at 640x384	~470 ms, from ~1,000 ms	One export; already planned
Hailo or Coral accelerator	10-30x	Hardware, approx. USD 70
Pi 5 with AI HAT (NPU)	Large; benchmarked in [8]	New board
Jetson Orin Nano	GPU inference; what FANS uses	Expensive
INT8 quantisation	~2x	Accuracy loss; needs calibration

Recommendation: do not change hardware. The contribution is the testbed and the measurement, and "the Pi 4B is too slow for real-time detection" is itself a finding supported by [7] and [8]. Substituting a Jetson would remove the very constraint the experiment set out to measure. The NCNN 640 export is worth doing: it is free and completes the results table.

Suggested wording for the report

"The use of ns-3 TapBridge to attach physical hardware to an emulated network follows the approach documented in the ns-3 model library [1] and applied in prior UAV co-simulation work such as CORNET [3] and FANS [5]. Unlike those systems, the present testbed separates the sensor and radio paths, so that only traffic which would physically traverse a radio is emulated — a distinction which also keeps ns-3 within the traffic regime in which its real-time mode remains accurate [6]." — adapt as needed

Sources

- [1] Emulation Overview — ns-3 Model Library
<https://www.nsnam.org/docs/models/html/emulation-overview.html>
- [2] HOWTO make ns-3 interact with the real world
https://www.nsnam.org/wiki/HOWTO_make_ns-3_interact_with_the_real_world
- [3] CORNET: A Co-Simulation Middleware for Robot Networks
<https://ece.iisc.ac.in/~parimal/papers/2020/comsnets.pdf>
- [4] FlyNetSim: An Open Source Synchronized UAV Network Simulator based on ns-3 and Ardupilot
<https://ar5iv.labs.arxiv.org/html/1808.04967>
- [5] Hardware Implementation of FANET Using FANS (Proc. ns-3 Conference 2025)
<https://dl.acm.org/doi/10.1145/3747204.3747223>
- [6] Performance Evaluation of ns-3 Real-Time Emulation
https://www.researchgate.net/publication/390270733_Performance_Evaluation_of_ns-3_Real-Time_Emulation
- [7] UAV detection with YOLO on a standalone Raspberry Pi 5 system
<https://ceur-ws.org/Vol-3970/PAPER1.pdf>
- [8] Review of large YOLOv8 and RT-DETR energy efficiency on edge devices for real-time detection
<https://www.nature.com/articles/s41598-026-46453-6>
- [9] Communication-Computation Trade-Off in Resource-Constrained Edge Inference
<https://arxiv.org/pdf/2006.02166>